

Master Interview Walkthrough, Code Guide & Future Roadmap

Candidate: Muhammad Aqib | Position: AI Agent Engineer | Interviewer: Alessandro Colford (isthispossible.ai)

1. THE 20-MINUTE GAME PLAN & TIME MANAGEMENT

20 minutes passes very quickly on a screen share. Keep strictly to this structure so you show the product, code, future roadmap, and tests comfortably.

Time Window	Stage	Key Objective & What You Are Doing
0:00 - 2:30	Strategic Intro	Hook Alessandro with the \$1.4M revenue leak. Explain why QuoteFlow was prioritized over marketing.
2:30 - 8:30	Live Deployed App	Show field notes input, Gemini AI extraction, confidence scores, edit drawer, PDF download, and Slack alert.
8:30 - 13:00	Code & Architecture	Switch to VS Code. Walk through models.py, llm.py, main.py, and db_fake.py test isolation.
13:00 - 16:30	Future Roadmap	Explain the 5-Agent Strategy, Dollar-Leverage rankings, Vector RAG scaling, and why marketing was rejected.
16:30 - 20:00	Tests & Q&A	Run pytest (62 tests in 10s). Answer questions calmly and ask Alessandro about client deployment challenges.

2. VERBATIM STEP-BY-STEP TALKING SCRIPT (PLAIN ENGLISH)

Step 1: The Intro (Under 2 minutes)

[WHAT TO SAY: Opening Hook]

"Hi Alessandro, great to meet you! Before jumping into the live app, let me give you the quick business rationale behind QuoteFlow Pro. When analyzing Marcus's landscaping business, he averages \$28,000 per project across roughly 150 jobs a year. His constraint is NOT marketing -- his ROAS is 4.5x and he cannot keep up with current leads. His real bottleneck is that after site walks, notes sit in notebooks for 6 to 9 days because Marcus is the only person pricing them manually in Excel. During that delay, 35 to 40% of clients go with faster competitors, costing over \$1.4M a year. QuoteFlow Pro solves this exact interpretation bottleneck: it turns raw notes into catalog-priced proposals in seconds, while keeping Marcus in control as the final approver."

Step 2: Live App Demo

[WHAT TO SAY: Intake Page (<http://localhost:8000>)]

"Here is the intake screen. An estimator simply types or pastes raw site-walk notes. Notice how messy these notes can be: shorthand abbreviations, mixed units like square feet and linear feet, and special notes like HOA permits. Let's hit 'Generate Proposal Draft'."

[WHAT TO SAY: Proposals Dashboard (/proposals)]

"In about two seconds, Gemini Flash extracted the scope and mapped every item to our 200+ item pricing catalog. Look at four key things here: First, accurate SKU matching with real unit prices. Second, AI Confidence Ratings: each line is tagged High, Medium, or Low with a plain-English reason so Marcus only needs to double-check uncertain items. Third, because the total exceeds \$30,000, it automatically flagged '3D Render Required'. Fourth, it captured the HOA requirement under Special Conditions."

[WHAT TO SAY: Interactive Editor & PDF Export]

"AI should never be an uncontrollable black box. If Marcus wants to adjust a quantity or add another item, he clicks Edit. He can change quantities or prices and hit Save -- the subtotal instantly recalculates. Next, if we click 'Download PDF', our built-in PDF engine generates a clean, branded proposal draft complete with scope summary, itemized tables, and legal terms ready for the homeowner."

[WHAT TO SAY: Approval & Slack Automation]

"When Marcus clicks 'Approve', it updates the status and triggers a Slack webhook. Because this quote exceeded \$30,000, the Slack message automatically alerts Carlos: 'Attention @Carlos - 3D Render Required!' so Carlos can begin 3D modeling immediately without Marcus having to write an email."

3. LINE-BY-LINE CODE BREAKDOWN IN PLAIN ENGLISH

Why app/models.py exists: LLMs output unpredictable text. Pydantic is our bouncer. It guarantees that whatever Gemini outputs strictly conforms to clean numbers, text, and structures before touching the database.

`class LineItem(BaseModel):`

--> In Plain Words: Blueprint for a single billable line item in a proposal (like Pavers or Sod).

`pricing_item_id: int`

--> In Plain Words: The exact database ID matching our Supabase pricing table SKU.

`name: str`

--> In Plain Words: The human-readable name of the catalog item.

`quantity: float = Field(gt=0)`

--> In Plain Words: 'float' allows decimals (2.5 tons). 'gt=0' means Greater Than 0. Prevents zero or negative quantities.

`unit_price: float = Field(gt=0)`

--> In Plain Words: Catalog unit price. Guarded with gt=0 to prevent accidental \$0 pricing.

`line_total: float = Field(gt=0)`

--> In Plain Words: Quantity times unit price. Guaranteed positive dollar value.

confidence: Literal["high", "medium", "low"] = "high" --> In Plain Words: Forces the AI to pick only one of these three exact words.
confidence_reason: Optional[str] = None --> In Plain Words: Short 1-sentence reason why that confidence was assigned.
class ParsedProposal(BaseModel): --> In Plain Words: The master contract for the entire quote that Gemini must satisfy.
client_name: str --> In Plain Words: The client or homeowner's name.
line_items: list[LineItem] = Field(min_length=1) --> In Plain Words: Must have at least 1 item. AI cannot generate a useless empty proposal.
subtotal: float = Field(ge=0) --> In Plain Words: Total dollar sum. Must be Greater than or Equal to 0.
notes_summary: str --> In Plain Words: 1-to-2 sentence plain summary so Marcus understands the job at a glance.
special_conditions: list[str] = [] --> In Plain Words: Non-billable flags like HOA approval, permits, or gate codes.
raw_notes: str = Field(min_length=10) --> In Plain Words: In SubmitNotesRequest: Blocks accidental empty submissions like 'hi' or 'ok'.
UpdateProposalRequest (all Optional) --> In Plain Words: Allows Marcus to update just 1 field in the edit drawer without resending everything.

What is 'Native JSON Mode' and Why Are We Doing It?

Normal LLMs talk like humans: 'Sure! Here is your quote: ```json ... ``` Hope this helps!'. If an LLM outputs markdown fences or chatting text, json.loads() crashes. Native JSON Mode (response_mime_type='application/json') constrains Gemini at the model level so every single token from the first bracket { to the last } is 100% pure JSON syntax -- zero markdown, zero chatting, zero crashes.

What is the Purpose of the 62 Tests? Why are we testing?

- Tests are automated checks that act like a picky quality-assurance robot. They prove the software does not break under pressure:
- Model Tests: Prove that negative prices or zero quantities are blocked by Pydantic.
 - LLM Guardrail Tests: Prove that if Gemini fails or returns garbage, the app saves it as a draft with parse_error instead of crashing.
 - DB & API Tests: Prove that saving, approving, rejecting, and CSV export work without a live database via conftest.py.
 - Slack & PDF Tests: Prove that quotes >\$30k tag @Carlos and PDF quote bytes are valid.

4. THE FUTURE ROADMAP & THE 5-AGENT STRATEGY

In STRATEGY.md, I ranked 5 AI agents by Dollar Leverage (recoverable revenue), not just what Marcus said out loud:

[WHAT TO SAY: Explaining the 5-Agent Roadmap] <i>"Alessandro, beyond this proposal tool, I mapped out a 5-agent ecosystem ranked strictly by ROI:</i> 1. Quote & Proposal Accelerator (P0 - Built): Solves the 7-day quote lag, recovering \$1.4M/year. 2. Post-Sign Milestone Tracker (P1): 8 to 12 projects sit stalled in HOA permits and deposit payments, delaying \$300K+ in cash flow. This agent auto-tracks permit portals and sends friendly nudges to homeowners and Jenna. 3. Build-Progress Update Agent (P2): When work starts, silence for 4 days makes clients anxious. This agent listens to CompanyCam photo uploads and drafts personal Marcus-voiced progress updates in 1 click. 4. Closed-Lost Reactivation Agent (P3): 1,400 past leads sit dormant in GoHighLevel CRM. Re-engaging them with personalized check-ins at a 2% close rate unlocks \$784K in latent revenue with zero ad spend. 5. Lead Pre-Qualification Voice/SMS Agent (P4): Filters tire-kickers before they take slots on Marcus's calendar." <i></i>

Why I Explicitly Rejected the Marketing Agent

Marcus mentioned marketing as his #4 priority. But he stated twice that ROAS is already 4.5x and he cannot keep up with current leads. Building a marketing agent generates leads he can't fulfill and makes the quote bottleneck worse. It is a clean, data-backed 'No'.

Technical Scaling Roadmap (Catalog > 500 items)

[WHAT TO SAY: Scaling from In-Context to Vector RAG] <i>"Right now, feeding all 200 catalog SKUs into Gemini takes ~3,500 tokens. It costs \$0.0003 and takes 1.5s. But if Marcus expands to 2,000 SKUs, putting the whole catalog into the prompt causes latency spikes and LLM attention drift. The production solution is a Two-Stage Vector RAG: store SKUs in Supabase using pgvector and embeddings. When field notes come in, we query the top 20 relevant items via semantic search, and feed only those into Gemini."</i>
--

5. ANTICIPATED QUESTIONS & BULLETPROOF SIMPLE ANSWERS

Q: Why Gemini Flash instead of GPT-4o or Claude Sonnet?

Answer: Three reasons: Speed, Cost, and Accuracy. Flash responds in 1.5 to 2 seconds, giving estimators a snappy real-time experience. For structured catalog mapping, heavy models like GPT-4o are 10 to 15 times more expensive with zero noticeable quality difference. Flash gives 99%+ schema

adherence when backed by Pydantic.

Q: What happens if an estimator mentions an item not in the catalog (e.g. exotic pizza oven)?

Answer: The prompt explicitly commands the AI never to invent prices or fake SKUs. If there is no reasonable match, it omits the item. In the UI, Marcus sees the raw notes side-by-side with the quote and can click Edit to manually add a custom item before approving.

Q: Why keep Marcus in the loop? Why not email the quote to the homeowner automatically?

Answer: Because this is high-ticket construction (\$28K average job). An unverified AI quote with a missing retaining wall or wrong square footage could cost Marcus \$10,000 out of pocket. Keeping Marcus as a 30-second reviewer gives him 10x speed while keeping 100% human quality control.

Q: How does the \$30,000 render threshold work?

Answer: In main.py, RENDER_THRESHOLD defaults to 30000. If the subtotal exceeds that, needs_render is set to True. This adds an alert badge in the UI and PDF, and instructs slack.py to mention @Carlos so he immediately starts 3D modeling without waiting for an email.

Q: How did you ensure reliable cloud deployment?

Answer: Cloud platforms like Render or Railway often pass the PORT environment variable with quotes or whitespace, which crashes standard Uvicorn. I built uvicorn_patch.py and start.sh with regex sanitization and fallback to port 8000, plus a multi-stage non-root Dockerfile.

6. PRE-CALL SETUP CHECKLIST

- 1. Open Browser: Tab 1 (<http://localhost:8000>), Tab 2 (/proposals), Tab 3 (Downloaded PDF sample).
- 2. Open VS Code: Have models.py or main.py open. Make sure terminal is zoomed in (Ctrl + +).
- 3. Test Run pytest: Run `$env:APP_ENV='test'; pytest` once before the call to make sure it prints '62 passed'.
- 4. Turn Off Notifications: Close WhatsApp, Discord, Slack, and mute phone notifications.
- 5. Keep this PDF Open on your second screen or phone for quick reference while talking!